WorldSkills Cloud Computing 프로젝트 트랙 CFP (초안)

2026-09-20 · 송무현

1. 취지와 배경

이번 트랙은 각 선수가 스스로 정의한 문제를 클라우드로 해결하는 프로젝트를 제안하고, 약 10주 동안 직접 lead하는 훈련입니다. 결과물만큼 진행 중에 겪은 문제, 판단 근거, 목표 대비 달성 과정을 기록하는 것을 핵심 산출물로 봅니다.

지금까지의 훈련은 특정 AWS service나 과제 유형을 반복 숙달하는 방식이었습니다. 경기에 필요한 속도와 정확도는 올라가지만, 다음 경험은 부족했습니다.

- 요구사항이 모호한 상태에서 목표와 scope를 스스로 정하는 경험
- 여러 설계 선택지 사이에서 trade-off를 판단하고 근거를 남기는 경험
- 예상하지 못한 장애를 진단하고 복구하는 경험
- 훈련 결과를 경기 이후 진학·취업에 쓸 수 있는 portfolio로 남기는 경험

기대 효과는 세 가지입니다.

- **경기 역량**: architecture 설계, IaC, troubleshooting, 비용·보안 고려를 실제 workload 위에서 통합적으로 연습
- **프로젝트 역량**: scope 설정, 일정·리스크 관리, 기술 의사결정 문서화
- **Portfolio**: 공개 가능한 repository, architecture diagram, engineering log, postmortem, 최종 발표 자료

2. 참가 대상과 형식

훈련 중인 선수 4명이 각자 다른 주제의 1인 1프로젝트로 참여하고, 선수는 목표 설정부터 최종 발표까지 책임지는 Project Lead가 됩니다.

역할	담당	책임
Project Lead	선수 본인	목표·scope 정의, 설계·구현, 일정 관리, 기록, 발표
Mentor	코치	제안서 심사, 주간 check-in, 방향 조정, 비용·보안 승인
Peer Reviewer	다른 선수 2명 (순환 배정)	제안서·설계·최종 결과물 리뷰와 개선 의견 제시

프로젝트 하나에는 반드시 reviewer 2명이 배정되고, 각 선수는 다른 두 명의 프로젝트를 리뷰합니다. 배정은 순환 방식입니다(1번 → 2·3번, 2번 → 3·4번, 3번 → 4·1번, 4번 → 1·2번).

3. 주제 트랙

아래 7개 트랙 중 하나를 골라 주제를 자유롭게 제안합니다. 트랙마다 적힌 주제는 방향과 난이도를 보여주기 위한 예시일 뿐이며, 예시와 다른 주제나 여러 트랙을 섞은 주제도 환영합니다. 설계와 구현 방법은 선수가 정하고, AI Agent 활용은 선택이며 F(FDE)만 Agent integration이 핵심입니다.

트랙	한 줄 요약	면접에서 특히 강한 직무
A. Cost Optimization (FinOps)	비용을 성능 손해 없이 줄이기	Cloud engineer, FinOps, SRE
B. AIOps	장애 탐지·원인 분석·복구 자동화	SRE, 운영 엔지니어
C. Chaos Engineering	장애를 주입해 약점을 찾고 고치기	SRE, 인프라 엔지니어
D. Platform Engineering	self-service 내부 플랫폼 만들기	DevOps, Platform engineer
E. DevSecOps	보안 검사와 대응을 pipeline에 넣기	Cloud security, DevSecOps
F. Forward Deployed Engineering	고객 현장의 문제를 풀고 AI Agent를 도입하기	FDE, Solutions engineer
G. Multi-tenant SaaS	여러 고객이 안전하게 나눠 쓰는 서비스 만들기	Backend engineer, Cloud architect

A. Cost Optimization (FinOps)

비용 데이터를 분석해 낭비를 찾고, 구조를 바꿔 성능 손해 없이 비용을 줄이는 트랙입니다.

예시 주제 (이 밖의 주제도 가능)

- **Spot 100% 운영**: 상시 트래픽이 있는 web 서비스의 compute를 전부 Spot으로 옮기면서 가용성 99.9%, p95 300 ms를 지킨다. 여러 AZ에서 Spot 회수가 동시에 일어나도 견뎌야 한다.
- **비용 50% 절감, 성능 동일**: 이미 rightsizing이 끝난 3-tier 서비스의 월 비용을 절반으로 줄이되 p95 latency와 처리량은 유지한다. architecture 자체를 바꿔야 하는 수준이다.
- **비용 이상 1시간 내 차단**: 여러 AWS 계정에서 비용 급증(잘못된 loop, 유출된 key로 띄운 instance 등)을 1시간 안에 탐지하고 원인 리소스까지 특정해 막는다. 청구 데이터는 몇 시간 늦게 갱신된다는 제약이 있다.

최종 지표 (예시)

- 정량: 월 비용 절감률, 절감 전후 p95 latency·가용성, 비용 이상 탐지·차단까지 걸린 시간
- 정성: 절감 결정별 trade-off 분석, 비용 구조를 설명하는 dashboard·보고서

B. AIOps

장애를 빨리 알아채고, 원인을 좁히고, 사람 개입 없이 복구하는 운영 자동화 트랙입니다.

예시 주제 (이 밖의 주제도 가능)

- **MTTR 5분**: microservice 5개 이상으로 구성된 서비스에 사전에 알려주지 않은 장애 10종(배포 결함, DB connection 고갈, 의존 서비스 지연 등)을 주입했을 때, 탐지부터 복구까지 평균 5분 이내를 달성한다.
- **처음 보는 장애의 원인 찾기**: 기존 알람·runbook에 없는 유형의 장애가 났을 때 원인 서비스와 원인 변경(commit, 설정)을 자동으로 좁혀, 상위 3개 후보 안에 정답이 들어가는 비율 80% 이상을 만든다.
- **Self-healing 배포**: 배포 직후 품질 저하를 사람 개입 없이 감지해 자동 rollback하되, 정상 배포를 잘못 되돌리는 비율은 5% 이하로 유지한다.

최종 지표 (예시)

- 정량: MTTD·MTTR, 원인 후보 적중률, 자동 복구 비율, rollback 오탐률
- 정성: runbook과 postmortem의 품질, 처음 보는 장애에 대한 대응 절차

C. Chaos Engineering

"장애가 나면 어떻게 되나요?"를 추측이 아니라 통제된 실험으로 답하고, 드러난 약점을 고치는 트랙입니다.

예시 주제 (이 밖의 주제도 가능)

- **Region 장애 생존**: 한 Region 전체를 쓸 수 없게 돼도 다른 Region에서 RTO 15분, RPO 1분 이내로 서비스를 이어간다. 전환 후 데이터 정합성까지 검증해야 한다.
- **Gray failure 격리**: instance가 죽지 않고 "느려지기만" 하는 부분 장애(일부 요청만 지연, packet loss)를 주입하고, 시스템이 스스로 격리해 영향받는 요청을 1% 미만으로 만든다.
- **연쇄 장애 차단**: 의존 서비스 하나의 지연이 retry storm으로 번져 전체가 무너지는 현상을 재현하고, 평소 부하의 3배에서도 장애가 번지지 않게 만든다.

최종 지표 (예시)

- 정량: 개선 전후 RTO·RPO·error rate, 장애 영향을 받은 요청 비율
- 정성: 실험별 가설·결과·개선 기록, 발견한 약점과 설계 변경 근거

D. Platform Engineering

개발자가 인프라를 몰라도 표준화된 환경을 스스로 받게 하는 내부 플랫폼을 만드는 트랙입니다.

예시 주제 (이 밖의 주제도 가능)

- **10분 안에 production-ready 환경**: 개발자가 요청 하나로 repository, CI/CD, monitoring, 보안 기준이 갖춰진 서비스 환경을 10분 안에 받는다. 만들어진 모든 환경은 같은 보안·비용 기준을 자동으로 만족해야 한다.
- **Zero-drift 인프라**: 여러 계정·환경에서 누군가 console로 바꾼 설정(drift)을 10분 안에 탐지하고, IaC 상태로 되돌리거나 변경을 Pull Request로 제안한다.
- **무중단 Kubernetes upgrade**: 여러 workload가 도는 EKS cluster를 서비스 중단 없이 두 minor version 연속 upgrade한다 (control plane, node, add-on 포함).

최종 지표 (예시)

- 정량: 환경 생성 시간, drift 탐지·복구 시간, upgrade 중 error rate
- 정성: 실제 사용자(다른 선수) 피드백, 표준·guardrail 문서

E. DevSecOps

보안을 배포 뒤 점검이 아니라 pipeline 안의 자동 검사와 자동 대응으로 만드는 트랙입니다.

예시 주제 (이 밖의 주제도 가능)

- **Access key 유출 5분 대응**: public repository에 AWS access key가 유출됐다고 가정하고, 탐지부터 key 비활성화, 그 key로 한 행동 추적, 피해 리소스 격리·보고까지 5분 안에 자동 처리한다.
- **자동 least privilege**: 서비스가 실제로 쓰는 권한만 남도록 IAM policy를 자동으로 축소하되, 기능이 하나도 깨지지 않았음을 test로 증명한다.
- **Supply chain 보안**: container image가 build부터 배포까지 변조되지 않았음을 서명과 SBOM으로 보장하고, 서명 없는 image는 cluster에서 실행되지 않게 한다.

최종 지표 (예시)

- 정량: 탐지→조치 시간, 줄인 IAM 권한 수, 차단한 미서명 image 수
- 정성: 위협 모델(threat model), 대응 playbook과 사후 보고서

F. Forward Deployed Engineering (FDE)

FDE는 고객 현장에 들어가 고객의 문제를 풀고, 고객 업무에 AI Agent를 붙여 실제로 쓰이게 만드는 직무입니다. 이 트랙은 AI Agent integration이 핵심이고, agent는 반드시 AWS 위에서 운영하며 비용·권한·데이터 위치를 선수가 책임집니다. 품질은 고객의 실제 요청으로 만든 평가 set으로 증명합니다.

예시 주제 (이 밖의 주제도 가능)

- **교무실 학사 안내 agent**: 학교 규정, 공지, 가정통신문을 근거로 학생·학부모 문의에 답하는 agent를 AWS에 구축해 학교 메신저·메일에 연결한다. 실제 문의 100건 기준 정답률 90% 이상, 불확실하면 담당 선생님에게 넘기고, 학생 개인정보는 학교 AWS 계정 밖으로 나가지 않으며 월 운영비 $30 이하를 지킨다.

- **실습실 클라우드 자원 agent**: 전공 수업·동아리 학생이 자연어로 실습 환경을 신청·연장·반납하면, agent가 학교 AWS 계정에서 IaC로 환경을 실제로 만들고 회수한다. 예산·권한 규칙을 어기는 생성은 0건이어야 하고, agent의 모든 행동은 CloudTrail과 감사 log로 추적되어야 한다.
- **지역 가게 고객지원 agent**: 동네 온라인 가게의 주문 데이터를 AWS로 옮기고, 문의 메일·메신저에 agent를 연결해 배송·환불 답변 초안을 쓰고 사장님 승인 후 발송한다. 응대 시간 50% 단축, 개인정보 노출 0건, 문의가 평소의 10배로 몰려도 비용과 응답 시간이 유지되어야 한다.

외부 고객을 구하기 어렵다면 코치나 학교 선생님이 고객 역할을 맡습니다.

최종 지표 (예시)

- 정량: 평가 set 정답률, escalation 비율, 고객 업무 시간 절감, 요청당 AWS 비용
- 정성: 고객 인터뷰·피드백, 고객이 직접 운영할 수 있는 수준의 인계 문서

G. Multi-tenant SaaS

여러 고객이 한 서비스를 나눠 쓰면서도 서로의 데이터와 성능에 영향을 주지 않게 만드는 트랙입니다.

예시 주제 (이 밖의 주제도 가능)

- **Noisy neighbor 차단**: tenant 1,000개가 한 서비스를 쓰는 상황을 부하로 재현하고, tenant 하나가 트래픽을 100배로 늘려도 다른 tenant의 p95 latency가 10% 이상 나빠지지 않게 한다.
- **무중단 tier 전환**: 무료 tenant(공유 환경)가 유료로 바뀌면 데이터 유실과 서비스 중단 없이 전용 환경으로 옮겨지게 한다.
- **tenant별 정확한 원가**: DB, cache, network 같은 공유 리소스까지 포함해 tenant별 원가를 계산하고, 실제 AWS 청구서와의 오차를 5% 이내로 맞춘다.

최종 지표 (예시)

- 정량: tenant 간 성능 격리 수치, tier 전환 중 downtime, 원가 계산 오차
- 정성: isolation 모델 선택 근거, tenant 온보딩·운영 절차 문서

Agent를 쓰는 경우 두 가지를 지킵니다.

- 삭제·배포·교정 같은 실제 변경은 사람 승인(human-in-the-loop)을 거친다.
- Agent 없는 기본 구현과 비교해 무엇이 나아졌는지 수치로 보인다. 면접에서는 “AI를 써봤다”보다 “Agent가 어디서 효과가 있고 어디서 없는지 안다”가 더 강합니다.

4. 제안서 제출 요건

제안서를 쓰기 전에 AI Agent를 먼저 공부합니다. Agent를 어디에 붙일 수 있고 어디서 틀리는지 알아야 주제와 목표를 제대로 정할 수 있기 때문입니다. 8장의 “제안서 작성 전 (필수)” 강좌 4개(약 7시간)를 먼저 수강하고, 제안서 4번 항목에 배운 내용을 어떻게 적용할지 적습니다.

제안서는 A4 3–4쪽(또는 같은 분량의 Markdown README)이며, 아래 7개 항목만 담으면 됩니다. 가장 중요한 것은 3번 최종 지표를 어떻게 측정할지입니다.

#	항목	작성 내용
1	제목 / 트랙	한 줄 제목과 선택한 트랙
2	문제 정의	어떤 문제를 왜 풀려는지 3–5문장
3	목표와 최종 지표	정량 지표 2–3개와 정성 지표 1–2개, 각각의 측정·판단 방법 (예: MTTR 5분 이내, postmortem 품질)
4	접근 방법	초기 architecture diagram 1장, 주요 service, AI Agent 활용 여부·범위와 사전 학습에서 배운 내용의 적용 계획
5	예상 비용	AWS Pricing Calculator 기준 월 예상 비용
6	예상 리스크	가장 걱정되는 기술적 어려움 2–3개
7	학습 계획	이 프로젝트를 위해 새로 공부해야 할 기술·서비스, 현재 수준, 필요한 이유, 공부 방법 (문서·강좌·실습). 공부한 내용은 engineering log에 기록

작성 예시는 `[예시 제안서] 훈련용 AWS 환경 비용 최적화` 를 참고합니다.

AI 사용 원칙: 제안서를 쓸 때 AI를 써도 됩니다. 단, 제안서에는 반드시 본인의 생각이 관철되어야 합니다.

- 문제 정의(2번)와 목표(3번)는 본인이 먼저 초안을 쓰고, AI는 그 뒤에 빠진 점과 반론을 찾는 데 씁니다.
- AI가 제안한 내용을 받아들이거나 버렸다면, 그 이유를 제안서 끝 “AI 사용 내역”에 한두 줄로 남깁니다.
- 제안서의 모든 문장은 코치와 reviewer 앞에서 본인 말로 설명할 수 있어야 합니다.

5. 프로젝트 관리 (Linear)

모든 프로젝트는 Linear로 관리하고, 코치와 reviewer 2명이 볼 수 있게 공유합니다. 실무 팀이 일하는 방식을 그대로 경험해 두면 면접에서 협업 방식을 구체적으로 설명할 수 있습니다.

- 선수별 프로젝트 하나를 Linear Project 하나로 만들고, 제안서의 성공 기준을 Milestone으로, 세부 작업을 Issue로 나눕니다.
- 2주 단위 Cycle로 작업을 정리하고, Cycle이 끝나면 완료·미완료와 그 이유를 남깁니다.
- 막힌 문제는 Issue로 만들고 engineering log 항목과 서로 link합니다.
- Reviewer 의견도 Issue로 받아 반영 여부를 기록합니다.
- GitHub integration을 켜고 Issue ID(예: `COST-12`)를 branch 이름에 넣어, Pull Request와 Issue 상태가 자동으로 연결되게 합니다.

- Linear MCP server를 AI coding tool에 연결하면 Issue를 읽고 갱신하는 일까지 도구가 도울 수 있습니다 (6장 참고).

6. AI Coding Tool 활용

Claude Code, OpenAI Codex, Cursor, GitHub Copilot 같은 AI coding tool은 적극적으로 쓰되, 선수가 설명할 수 없는 코드는 merge하지 않는 것이 원칙입니다. 목표는 "AI가 대신 만든 프로젝트"가 아니라 "AI를 어디에 어떻게 썼고 어디서 틀렸는지 설명할 수 있는 프로젝트"입니다.

단계	이렇게 쓴다	직접 확인할 것
설계	요구사항과 제약을 주고 architecture 대안 2–3개와 trade-off를 받아 ADR 초안으로 쓴다	최종 결정과 근거는 본인이 쓴다
IaC 작성	CloudFormation·Terraform 초안을 만들고 cfn-lint, terraform validate, Checkov로 검증 후 적용한다	IAM policy의 `*` 권한, public 설정, 삭제 정책
Troubleshooting	error message·log·재현 절차를 주고 원인 가설 목록을 받는다	가설은 직접 검증하고, 맞은 가설과 틀린 가설을 log에 남긴다
테스트·문서	부하 테스트 script, README, runbook 초안을 만든다	실제로 실행해 본 결과와 맞는지
코드 리뷰	Pull Request를 AI에게 먼저 리뷰받고 사람 reviewer에게 넘긴다	AI 지적 중 반영한 것과 버린 것의 이유

잘 쓰기 위한 세 가지 습관을 둡니다.

- **Context 파일**: repository에 `CLAUDE.md`나 `AGENTS.md` 같은 project context 파일을 두고 architecture, region, tag 규칙, 금지 사항을 적어 둡니다. 도구가 매번 같은 전제로 작업합니다.
- **권한 분리**: AI 도구에는 read-only 또는 sandbox 계정의 credential만 주고, 배포·삭제 명령은 사람이 확인한 뒤 실행합니다.
- **사용 기록**: AI가 크게 기여한 지점과 틀렸던 지점을 engineering log에 `AI` tag로 남깁니다. 최종 발표 Q&A에서는 생성된 코드를 선수가 직접 설명하는지 확인합니다.

7. 필수 산출물과 portfolio 활용

목표를 다 달성하지 못해도 괜찮습니다. 미달성 원인을 분석하고 조정한 과정 자체가 평가 대상이며, portfolio에서도 가장 설득력 있는 부분입니다.

산출물	내용	시점
Git repository	IaC 코드, application 코드, 재현 방법이 있는 README	상시
Project board	Linear의 Project, Milestone, Issue, Cycle 기록	상시
Architecture diagram	초기안과 최종안 모두 (변화 과정을 보여주기 위해)	설계 시, 종료 시
Engineering log	날짜별 작업, 발생한 문제, 시도한 방법, 결과	매주
ADR (Architecture Decision Record)	주요 기술 결정마다 맥락 / 선택지 / 결정 / 결과	결정 시점마다
Postmortem	장애 최소 1건, blameless 형식 (타임라인, 영향, root cause, 재발 방지)	장애 발생 후
최종 보고서	성공 기준별 달성 여부와 이유, 비용 결산, 회고	종료 시
발표 자료 & demo 영상	15분 발표, 3분 이내 demo 영상	최종 발표

Engineering log는 아래 형식으로 남깁니다. 문제 하나에 가설·시도·결과가 있어야 portfolio로 옮길 수 있습니다.

```
## 2026-10-14
- 한 일: ALB + ECS service 배포
- 문제: target이 unhealthy 상태를 반복
- 가설/시도:
  1. Security Group inbound 확인 → 정상
  2. health check path 확인 → / 가 302 반환
- 결과: path를 /healthz로 바꾸고 matcher 200 설정 후 해결
- 소요 시간: 1.5h
- 배운 점: health check는 redirect를 실패로 본다
```

Portfolio에는 최종 보고서를 1–2쪽으로 줄여 문제 → 접근 → 결과 수치 → 배운 점 순서로 올립니다. 이력서 bullet은 “EKS 서비스에 HPA를 적용해 부하 10배에서 p95 180 ms 유지, 유휴 비용 35% 절감”처럼 행동과 수치를 함께 씁니다(예시 형식).

8. 참고 자료: Claude Academy

Anthropic의 무료 교육 과정인 Claude Academy에서 이 트랙에 바로 쓰이는 강좌를 시점별로 골랐습니다. “제안서 작성 전 (필수)” 4개는 모든 선수가 제안서를 쓰기 전에 수강하고, 나머지는 진행 상황에 따라 고릅니다.

시점	강좌	분량	연결되는 내용
제안서 작성 전 (필수)	Claude Code 101	12 lessons, 1.5시간	AI coding tool 기본 사용법 (6장)
제안서 작성 전 (필수)	AI Capabilities and Limitations	13 lessons, 3.5시간	LLM·agent가 잘하는 것과 틀리는 지점
제안서 작성 전 (필수)	Introduction to Model Context Protocol	10 lessons, 1시간	agent를 AWS·Linear 같은 도구에 연결하는 방법
제안서 작성 전 (필수)	Building Effective Human-Agent Teams (beta)	5 lessons, 45분	사람과 agent의 역할 분담, 승인 단계 설계
구현 중	Claude Code in Action	9 lessons, 1시간	긴 작업을 맡기고 결과를 검증하는 방법
구현 중	Introduction to Agent Skills	6 lessons, 1시간	반복 작업을 skill로 묶기
구현 중	Introduction to Subagents	4 lessons, 45분	리뷰·테스트 작업을 subagent로 나누기
구현 중	The AI-Native SDLC Playbook	14 lessons, 1시간	AI를 포함한 개발 프로세스 (5장)
Agent 구현 시	Claude with Amazon Bedrock	65 lessons, 8시간	AWS 안에서 agent 구현, 실습은 저가 모델 위주로 (아래 참고)
선택	AI Fluency for Students	5 lessons	AI와 협업하는 원칙과 사용 기록 습관

Amazon Bedrock 실습 시 비용과 도구 연동

- **저가 모델부터**: 개발·실습은 Claude Haiku 4.5, Amazon Nova Micro, gpt-oss-20b 같은 저가 모델로 시작하고, 평가 set에서 품질이 부족할 때만 상위 모델로 올립니다. 예를 들어 Nova Micro는 1M token당 입력 $0.08, 출력 $0.32입니다 (Bedrock pricing).
- **Claude Code on Bedrock**: `CLAUDE_CODE_USE_BEDROCK=1`로 선수 AWS 계정을 통해 쓸 수 있습니다. 모델을 고정하지 않으면 비싼 기본 모델이 쓰이므로 `ANTHROPIC_MODEL`을 Sonnet이나 Haiku로 고정합니다 (설정 문서).
- **OpenAI Codex on Bedrock**: `~/.codex/config.toml`에 `model_provider = "amazon-bedrock"`을 넣으면 AWS credential로 Bedrock의 OpenAI 모델을 씁니다 (설정 문서). 같은 작업을 Claude Code와 Codex로 각각 해 보고 결과·비용을 비교하는 것도 좋은 portfolio 소재입니다.

Bedrock 비용 한도 설정 (필수)

Bedrock은 호출한 만큼 과금되고 AWS Budgets가 보는 청구 데이터는 하루 한 번 이상 갱신되므로, 알림만으로는 늦습니다. agent나 AI coding tool을 Bedrock에 연결하기 전에 아래 네 가지를 먼저 설정하고 코치에게 확인받습니다.

1. **호출 가능한 모델 제한**: IAM policy의 Resource를 저가 모델과 그 inference profile ARN으로만 한정해, 비싼 모델은 호출 자체가 안 되게 합니다. 여러 계정이면 SCP로 같은 제한을 겁니다.
2. **선수·프로젝트별 추적**: application inference profile을 만들고 `Owner`, `Project` tag를 붙여 누가 얼마나 썼는지 비용을 나눕니다 (문서).
3. **실시간 token 한도**: CloudWatch `AWS/Bedrock` namespace의 `InputTokenCount`·`OutputTokenCount` 지표로 일일 token 한도 알람을 만들고, 한도를 넘으면 Lambda가 Bedrock 호출을 거부하는 IAM policy를 붙이게 합니다 (지표 목록).
4. **월 예산 hard stop**: AWS Budgets action으로 월 예산 80%에서 알림, 100%에서 Bedrock 호출을 거부하는 IAM policy를 자동 적용합니다 (Budgets actions).

Claude Code·Codex처럼 스스로 작업을 반복하는 도구는 token을 빠르게 쓰므로, 3번의 일일 한도를 낮게 시작해 사용량을 보며 올립니다.

더 깊게 가려면 Model Context Protocol: Advanced Topics와 Building with the Claude API를 봅니다. 강좌 목록과 분량은 2026-09-20에 확인한 값입니다.